

Smart Legal Assistant for Indian Court Cases

Senior AI Research Engineer Implementation

This notebook implements a comprehensive legal NLP system using state-of-the-art models for:

- **Legal Document Summarization** (5 SOTA models)
- **Legal Question Answering** (5 specialized models)
- **Performance Benchmarking** with ROUGE, F1, and Exact Match metrics

Dataset Schema

```
{
  "judgment_text": "Full legal judgment text",
  "summary": "Ground truth summary",
  "qa_pairs": [
    {
      "question": "What was the verdict?",
      "answer": "The court ruled...",
      "question_type": "factual"
    }
  ]
}
```

Cell 1: Install Dependencies and Import Libraries

Installing all required packages for transformer models, evaluation metrics, and data processing.

```
# Install required packages
!pip install -q transformers datasets evaluate rouge_score accelerate sentencepiece protobuf
!pip install -q torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118
!pip install -q PyPDF2 tqdm pandas matplotlib seaborn

print("✓ All dependencies installed successfully!")
```

```
✓ All dependencies installed successfully!
```

```

# Import necessary libraries
import json
import torch
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm.auto import tqdm
from typing import List, Dict, Tuple
import warnings
warnings.filterwarnings('ignore')

# Transformers imports
from transformers import (
    AutoTokenizer,
    AutoModelForSeq2SeqLM,
    AutoModelForQuestionAnswering,
    pipeline
)

# Evaluation metrics
import evaluate
from rouge_score import rouge_scorer

# PDF processing
from PyPDF2 import PdfReader
from google.colab import files

# Set device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"\n🚀 Using device: {device}")
if torch.cuda.is_available():
    print(f"    GPU: {torch.cuda.get_device_name(0)}")
    print(f"    Memory: {torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")

```

```

🚀 Using device: cuda
GPU: Tesla T4
Memory: 15.83 GB

```

✓ Cell 2: Data Loading and Preprocessing

Loading the Real Indian Legal Judgments dataset and preparing it for model input.

```

# Sample dataset structure (Real Indian Legal Judgments v1.0)
# In production, load from: legal_dataset.json

```

```

SAMPLE_LEGAL_DATASET = [
    {
        "case_id": "SC_2019_001",
        "judgment_text": ""IN THE SUPREME COURT OF INDIA

```

```

CRIMINAL APPELLATE JURISDICTION
CRIMINAL APPEAL NO. 1234 OF 2019

```

```

STATE OF MAHARASHTRA ...APPELLANT
VERSUS
RAMESH KUMAR SHARMA ...RESPONDENT

```

```

JUDGMENT

```

This appeal arises from the judgment and order dated 15.03.2018 passed by the High Court of Bombay whereby the High Court ac

FACTS OF THE CASE:

The prosecution case is that on 10.01.2015, at approximately 8:30 PM, the deceased Vijay Kumar was found dead in his residen

1. Financial disputes between the deceased and respondent amounting to Rs. 50 lakhs
2. Presence of respondent's fingerprints at the crime scene
3. Recovery of the murder weapon from respondent's vehicle
4. Testimony of eyewitness who saw respondent leaving the deceased's residence at 8:45 PM

The Trial Court convicted the respondent under Section 302 IPC and sentenced him to life imprisonment. On appeal, the High C

LEGAL ANALYSIS:

We have carefully examined the evidence on record. The law is well-settled that in cases based on circumstantial evidence, t

In the present case, while the circumstantial evidence is strong, certain critical gaps exist:

1. The exact time of death could not be established beyond reasonable doubt
2. The eyewitness testimony was contradictory regarding the time of departure
3. No blood stains matching the deceased were found on the respondent's clothing
4. The murder weapon, though recovered, had no clear chain of custody documentation

Relying on the precedent set in *Sharad Birdhichand Sarda v. State of Maharashtra* (1984) 4 SCC 116, we hold that when two vie

CONCLUSION:

After thorough examination, we find that the prosecution has failed to prove guilt beyond reasonable doubt. The benefit of d

The appeal is dismissed. The judgment of the High Court acquitting the respondent is upheld. The respondent, if in custody,

Dated: 25th September 2019

(Justice A.K. Sikri)

(Justice S. Abdul Nazeer)""",

"summary": "The Supreme Court dismissed the State's appeal against the High Court's acquittal order in a murder case

```

"qa_pairs": [
    {
        "question": "What was the nature of the dispute between the deceased and the respondent?",
        "answer": "Financial disputes amounting to Rs. 50 lakhs",
        "question_type": "factual"
    },
    {
        "question": "Under which section of IPC was the respondent originally convicted?",
        "answer": "Section 302 IPC",
        "question_type": "factual"
    },
    {
        "question": "What was the final decision of the Supreme Court?",
        "answer": "The appeal was dismissed and the High Court's acquittal order was upheld",
        "question_type": "reasoning"
    },
    {
        "question": "Which precedent case was cited by the Supreme Court?",
        "answer": "Sharad Birdhichand Sarda v. State of Maharashtra (1984) 4 SCC 116",
        "question_type": "factual"
    },
    {
        "question": "What were the critical gaps in evidence identified by the Court?",
        "answer": "Uncertain time of death, contradictory eyewitness testimony, absence of blood stains on responden
    }
]
]

```

Function to load dataset

```
def load_legal_dataset(file_path: str = None) -> List[Dict]:
```

```

    """
    Load legal dataset from JSON file.

```

Args:

file_path: Path to legal_dataset.json (if None, uses sample data)

Returns:

List of legal case dictionaries

```
"""
```

```
if file_path:
```

```
    try:
```

```
        with open(file_path, 'r', encoding='utf-8') as f:
```

```
            dataset = json.load(f)
```

```
            print(f"✓ Loaded {len(dataset)} legal cases from {file_path}")
```

```
            return dataset
```

```
    except FileNotFoundError:
```

```
        print(f"⚠ File not found: {file_path}. Using sample dataset.")
```

```
        return SAMPLE_LEGAL_DATASET
```

```
else:
```

```
    print("i Using sample legal dataset (1 case)")
```

```
    return SAMPLE_LEGAL_DATASET
```

Load dataset

```
legal_dataset = load_legal_dataset()
```

Display dataset statistics

```
print("\n 📊 Dataset Statistics:")
```

```
print(f"    Total Cases: {len(legal_dataset)}")
```

```
if legal_dataset:
```

```
    avg_judgment_length = np.mean([len(case['judgment_text'].split()) for case in legal_dataset])
```

```
    avg_qa_pairs = np.mean([len(case['qa_pairs']) for case in legal_dataset])
```

```
    print(f"    Average Judgment Length: {avg_judgment_length:.0f} words")
```

```
    print(f"    Average QA Pairs per Case: {avg_qa_pairs:.1f}")
```

```
    print(f"\n    Sample Case ID: {legal_dataset[0]['case_id']}")
```

i Using sample legal dataset (1 case)

```

📊 Dataset Statistics:
Total Cases: 1

```

Average Judgment Length: 420 words
Average QA Pairs per Case: 5.0

Sample Case ID: SC_2019_001

Cell 3: Summarization Engine

Implementation of 5 state-of-the-art summarization models with comparison view and ROUGE evaluation.

```
# Define summarization models
SUMMARIZATION_MODELS = {
    "BART-Large-CNN": {
        "model_name": "facebook/bart-large-cnn",
        "description": "State-of-the-art abstractive summarizer",
        "max_length": 1024,
        "min_length": 100
    },
    "Pegasus-ArXiv": {
        "model_name": "google/pegasus-arxiv",
        "description": "Gap Sentence Generation for superior ROUGE scores",
        "max_length": 512,
        "min_length": 100
    },
    "Long-T5-TGlobal": {
        "model_name": "google/long-t5-tglobal-base",
        "description": "Long-document summarization with global attention",
        "max_length": 512,
        "min_length": 100
    },
    "Legal-LED": {
        "model_name": "nsi319/legal-led-base-16384",
        "description": "Fine-tuned for legal and legislative text",
        "max_length": 1024,
        "min_length": 100
    },
    "T5-Large": {
        "model_name": "google-t5/t5-large",
        "description": "Text-to-text transfer transformer",
        "max_length": 512,
        "min_length": 100
    }
}

class LegalSummarizationEngine:
    """
    Research-grade summarization engine for legal documents.
    Implements multiple SOTA models with comprehensive evaluation.
    """

    def __init__(self):
        self.models = {}
        self.tokenizers = {}
        self.rouge_scorer = rouge_scorer.RougeScorer(
            ['rouge1', 'rouge2', 'rougeL'],
            use_stemmer=True
        )

    def load_model(self, model_key: str):
        """
        Load a specific summarization model.

        Args:
            model_key: Key from SUMMARIZATION_MODELS dict
        """
        if model_key in self.models:
            return # Already loaded

        model_config = SUMMARIZATION_MODELS[model_key]
        model_name = model_config["model_name"]

        print(f"\n 🔄 Loading {model_key}...")
        print(f"    {model_config['description']}")

        try:
            # Load tokenizer and model
            tokenizer = AutoTokenizer.from_pretrained(model_name)
            model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
            model = model.to(device)
            model.eval()

            self.tokenizers[model_key] = tokenizer
```

```

        self.models[model_key] = model

        print(f"    ✓ Successfully loaded {model_key}")
    except Exception as e:
        print(f"    ✗ Failed to load {model_key}: {str(e)}")

def generate_summary(self, text: str, model_key: str,
                    max_summary_length: int = 150) -> str:
    """
    Generate summary using specified model.

    Args:
        text: Input legal judgment text
        model_key: Model to use for summarization
        max_summary_length: Maximum length of generated summary

    Returns:
        Generated summary text
    """
    if model_key not in self.models:
        self.load_model(model_key)

    model = self.models[model_key]
    tokenizer = self.tokenizers[model_key]
    config = SUMMARIZATION_MODELS[model_key]

    # Prepare input
    # For T5 models, add prefix
    if 't5' in config['model_name'].lower():
        text = f"summarize: {text}"

    # Tokenize with truncation
    inputs = tokenizer(
        text,
        max_length=config['max_length'],
        truncation=True,
        return_tensors="pt"
    ).to(device)

    # Generate summary
    with torch.no_grad():
        summary_ids = model.generate(
            inputs["input_ids"],
            max_length=max_summary_length,
            min_length=config['min_length'],
            num_beams=4,
            length_penalty=2.0,
            early_stopping=True
        )

    summary = tokenizer.decode(summary_ids[0], skip_special_tokens=True)
    return summary

def calculate_rouge(self, generated: str, reference: str) -> Dict[str, float]:
    """
    Calculate ROUGE scores.

    Args:
        generated: Generated summary
        reference: Ground truth summary

    Returns:
        Dictionary with ROUGE-1, ROUGE-2, ROUGE-L F1 scores
    """
    scores = self.rouge_scorer.score(reference, generated)
    return {
        'rouge1': scores['rouge1'].fmeasure,
        'rouge2': scores['rouge2'].fmeasure,
        'rougeL': scores['rougeL'].fmeasure
    }

def compare_all_models(self, judgment_text: str, ground_truth: str) -> pd.DataFrame:
    """
    Run all summarization models and compare results.

    Args:
        judgment_text: Input legal judgment
        ground_truth: Reference summary for evaluation

    Returns:
        DataFrame with summaries and ROUGE scores
    """

```

```

results = []

print("\n" + "="*80)
print("🔍 LEGAL DOCUMENT SUMMARIZATION - MODEL COMPARISON")
print("="*80)

for model_key in tqdm(SUMMARIZATION_MODELS.keys(), desc="Generating summaries"):
    # Generate summary
    summary = self.generate_summary(judgment_text, model_key)

    # Calculate ROUGE scores
    rouge_scores = self.calculate_rouge(summary, ground_truth)

    results.append({
        'Model': model_key,
        'Description': SUMMARIZATION_MODELS[model_key]['description'],
        'Generated_Summary': summary,
        'ROUGE-1': rouge_scores['rouge1'],
        'ROUGE-2': rouge_scores['rouge2'],
        'ROUGE-L': rouge_scores['rougeL'],
        'Summary_Length': len(summary.split())
    })

df = pd.DataFrame(results)
return df

# Initialize summarization engine
summarization_engine = LegalSummarizationEngine()
print("\n✓ Summarization Engine initialized successfully!")

```

✓ Summarization Engine initialized successfully!

```

# Run summarization comparison on sample case
sample_case = legal_dataset[0]

print("\n📄 ORIGINAL JUDGMENT TEXT (First 500 characters):")
print("-" * 80)
print(sample_case['judgment_text'][:500] + "...")

print("\n📄 GROUND TRUTH SUMMARY:")
print("-" * 80)
print(sample_case['summary'])

# Generate summaries from all models
summary_results = summarization_engine.compare_all_models(
    sample_case['judgment_text'],
    sample_case['summary']
)

# Display comparison table
print("\n\n" + "="*80)
print("📊 SUMMARIZATION RESULTS - COMPARISON TABLE")
print("="*80)

display_df = summary_results[['Model', 'ROUGE-1', 'ROUGE-2', 'ROUGE-L', 'Summary_Length']].copy()
display_df['ROUGE-1'] = display_df['ROUGE-1'].apply(lambda x: f"{x:.4f}")
display_df['ROUGE-2'] = display_df['ROUGE-2'].apply(lambda x: f"{x:.4f}")
display_df['ROUGE-L'] = display_df['ROUGE-L'].apply(lambda x: f"{x:.4f}")
print(display_df.to_string(index=False))

# Display generated summaries
print("\n\n" + "="*80)
print("📄 GENERATED SUMMARIES - DETAILED VIEW")
print("="*80)

for idx, row in summary_results.iterrows():
    print(f"\n\n{'-'*80}")
    print(f"Model: {row['Model']}")
    print(f>Description: {row['Description']}")
    print(f"ROUGE Scores - R1: {row['ROUGE-1']:.4f} | R2: {row['ROUGE-2']:.4f} | RL: {row['ROUGE-L']:.4f}")
    print(f"{'-'*80}")
    print(f"Generated_Summary: {row['Generated_Summary']}")

# Identify best model
best_model_idx = summary_results['ROUGE-L'].idxmax()
best_model = summary_results.loc[best_model_idx, 'Model']
best_rouge_l = summary_results.loc[best_model_idx, 'ROUGE-L']

print(f"\n\n{'-'*80}")
print(f"🏆 BEST PERFORMING MODEL: {best_model}")

```

```
print(f"    ROUGE-L Score: {best_rouge_l:.4f}")  
print(f"{'='*80}")
```


ORIGINAL JUDGMENT TEXT (First 500 characters):

IN THE SUPREME COURT OF INDIA

CRIMINAL APPELLATE JURISDICTION
CRIMINAL APPEAL NO. 1234 OF 2019

STATE OF MAHARASHTRA ...APPELLANT
VERSUS
RAMESH KUMAR SHARMA ...RESPONDENT

JUDGMENT

This appeal arises from the judgment and order dated 15.03.2018 passed by the High Court of Bombay whereby the High Court ac

FACTS OF THE CASE:

The prosecution case is that on 10.01.2015, at approximately...

GROUND TRUTH SUMMARY:

The Supreme Court dismissed the State's appeal against the High Court's acquittal order in a murder case under Section 302 I

LEGAL DOCUMENT SUMMARIZATION - MODEL COMPARISON

Generating summaries: 100% 5/5 [02:12<00:00, 28.84s/it]

Loading BART-Large-CNN...
State-of-the-art abstractive summarizer

Loading weights: 100% 511/511 [00:00<00:00, 850.46it/s, Materializing param=model.encoder.layers.11.self_attn_layer_norm.weight]

✓ Successfully loaded BART-Large-CNN

Loading Pegasus-ArXiv...
Gap Sentence Generation for superior ROUGE scores

pytorch_model.bin: 100% 2.28G/2.28G [00:19<00:00, 123MB/s]

model.safetensors: 100% 2.28G/2.28G [00:22<00:00, 275MB/s]

Loading weights: 100% 680/680 [00:01<00:00, 399.41it/s, Materializing param=model.shared.weight]

The tied weights mapping and config for this model specifies to tie model.shared.weight to model.decoder.embed_tokens.weight
The tied weights mapping and config for this model specifies to tie model.shared.weight to model.encoder.embed_tokens.weight
PegasusForConditionalGeneration LOAD REPORT from: google/pegasus-arxiv

Key	Status
model.decoder.embed_positions.weight	MISSING
model.encoder.embed_positions.weight	MISSING

Notes:

- MISSING :those params were newly initialized because missing from the checkpoint. Consider training on your downstream t

generation_config.json: 100% 280/280 [00:00<00:00, 6.46kB/s]

✓ Successfully loaded Pegasus-ArXiv

Loading Long-T5-TGlobal...
Long-document summarization with global attention

config.json: 100% 851/851 [00:00<00:00, 102kB/s]

spiece.model: 100% 792k/792k [00:00<00:00, 1.38MB/s]

tokenizer.json: 1.39M/? [00:00<00:00, 33.0MB/s]

pytorch_model.bin: 100% 990M/990M [00:05<00:00, 274MB/s]

Loading weights: 100% 297/297 [00:00<00:00, 897.95it/s, Materializing param=shared.weight]

generation_config.json: 100% 147/147 [00:00<00:00, 7.88kB/s]

✓ Successfully loaded Long-T5-TGlobal

model.safetensors: 100% 990M/990M [00:12<00:00, 212MB/s]

Loading Legal-LED...
Fine-tuned for legal and legislative text

config.json: 1.15k/? [00:00<00:00, 78.0kB/s]

tokenizer_config.json: 1.06k/? [00:00<00:00, 92.3kB/s]

vocab.json: 798k/? [00:00<00:00, 4.07MB/s]

merges.txt: 456k/? [00:00<00:00, 1.90MB/s]

special_tokens_map.json: 100% 772/772 [00:00<00:00, 82.0kB/s]

pytorch_model.bin: 100% 648M/648M [00:05<00:00, 146MB/s]

Loading weights: 100% 299/299 [00:00<00:00, 764.05it/s, Materializing param=lm_head.weight]

The tied weights mapping and config for this model specifies to tie led.shared.weight to lm_head.weight, but both are preser

Cell 4: Question Answering Engine

model.safetensors: 100%

648M/648M [00:04<00:00, 176MB/s]

Implementation of 5 specialized QA models with F1 and Exact Match evaluation

✓ Successfully loaded Legal-LED

```
# Define QA models
QA_MODELS = {
    "RoBERTa-SQuAD2": {
        "model_name": "deepset/roberta-base-squad2",
        "description": "Strong extractive QA baseline"
    },
    "BERT-Large-SQuAD2": {
        "model_name": "deepset/bert-large-uncased-whole-word-masking-squad2",
        "description": "Classical strong baseline for extractive QA"
    },
    "Legal-BERT": {
        "model_name": "nsi319/legal-bert-base-uncased",
        "description": "Domain-adapted for legal information extraction"
    },
    "FLAN-T5-Base": {
        "model_name": "google/flan-t5-base",
        "description": "Instruction-tuned generative QA"
    },
    "DeBERTa-v3": {
        "model_name": "microsoft/deberta-v3-base",
        "description": "Improved attention disentanglement"
    }
}
```

```
class LegalQAEEngine:
    """
    Research-grade Question Answering engine for legal documents.
    Supports both extractive and generative QA models.
    """

    def __init__(self):
        self.extractive_pipelines = {}
        self.generative_models = {}
        self.generative_tokenizers = {}

    def load_extractive_model(self, model_key: str):
        """
        Load extractive QA model (BERT, RoBERTa, DeBERTa, Legal-BERT).

        Args:
            model_key: Key from QA_MODELS dict
        """
        if model_key in self.extractive_pipelines:
            return

        model_config = QA_MODELS[model_key]
        model_name = model_config["model_name"]

        print(f"\n 🔄 Loading {model_key}...")
        print(f"    {model_config['description']}")

        try:
            qa_pipeline = pipeline(
                "question-answering",
                model=model_name,
                device=-1 if torch.cuda.is_available() else -1
            )
            self.extractive_pipelines[model_key] = qa_pipeline
            print(f"    ✓ Successfully loaded {model_key}")
        except Exception as e:
            print(f"    ✗ Failed to load {model_key}: {str(e)}")

    def load_generative_model(self, model_key: str):
        """
        Load generative QA model (FLAN-T5).

        Args:
            model_key: Key from QA_MODELS dict
        """
        if model_key in self.generative_models:
            return

        model_config = QA_MODELS[model_key]
        model_name = model_config["model_name"]

        print(f"\n 🔄 Loading {model_key}...")
        print(f"    {model_config['description']}")
```

```

try:
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
    model = model.to(device)
    model.eval()

    self.generative_tokenizers[model_key] = tokenizer
    self.generative_models[model_key] = model
    print(f"    ✓ Successfully loaded {model_key}")
except Exception as e:
    print(f"    ✗ Failed to load {model_key}: {str(e)}")

def answer_question_extractive(self, context: str, question: str,
                              model_key: str) -> Dict:
    """
    Answer question using extractive QA model.

    Args:
        context: Legal judgment text
        question: Question to answer
        model_key: Model to use

    Returns:
        Dictionary with answer and confidence score
    """
    if model_key not in self.extractive_pipelines:
        self.load_extractive_model(model_key)

    pipeline_model = self.extractive_pipelines[model_key]

    # Run QA pipeline
    result = pipeline_model(
        question=question,
        context=context,
        max_answer_len=100,
        handle_impossible_answer=True
    )

    return {
        'answer': result['answer'],
        'score': result['score'],
        'start': result.get('start', 0),
        'end': result.get('end', 0)
    }

def answer_question_generative(self, context: str, question: str,
                              model_key: str) -> str:
    """
    Answer question using generative QA model.

    Args:
        context: Legal judgment text
        question: Question to answer
        model_key: Model to use

    Returns:
        Generated answer text
    """
    if model_key not in self.generative_models:
        self.load_generative_model(model_key)

    model = self.generative_models[model_key]
    tokenizer = self.generative_tokenizers[model_key]

    # Format input for FLAN-T5
    input_text = f"question: {question} context: {context}"

    inputs = tokenizer(
        input_text,
        max_length=512,
        truncation=True,
        return_tensors="pt"
    ).to(device)

    with torch.no_grad():
        outputs = model.generate(
            inputs["input_ids"],
            max_length=100,
            num_beams=4,
            early_stopping=True
        )

```

```

        answer = tokenizer.decode(outputs[0], skip_special_tokens=True)
        return answer

def calculate_f1_score(self, predicted: str, ground_truth: str) -> float:
    """
    Calculate F1 score between predicted and ground truth answers.

    Args:
        predicted: Predicted answer
        ground_truth: Reference answer

    Returns:
        F1 score (0.0 to 1.0)
    """
    # Normalize strings
    pred_tokens = set(predicted.lower().split())
    truth_tokens = set(ground_truth.lower().split())

    if len(pred_tokens) == 0 or len(truth_tokens) == 0:
        return 0.0

    # Calculate precision and recall
    common_tokens = pred_tokens & truth_tokens

    if len(common_tokens) == 0:
        return 0.0

    precision = len(common_tokens) / len(pred_tokens)
    recall = len(common_tokens) / len(truth_tokens)

    # F1 score
    f1 = 2 * (precision * recall) / (precision + recall)
    return f1

def calculate_exact_match(self, predicted: str, ground_truth: str) -> int:
    """
    Calculate exact match score.

    Args:
        predicted: Predicted answer
        ground_truth: Reference answer

    Returns:
        1 if exact match, 0 otherwise
    """
    # Normalize and compare
    pred_normalized = predicted.lower().strip()
    truth_normalized = ground_truth.lower().strip()

    return 1 if pred_normalized == truth_normalized else 0

def evaluate_all_models(self, context: str, qa_pairs: List[Dict]) -> pd.DataFrame:
    """
    Evaluate all QA models on given question-answer pairs.

    Args:
        context: Legal judgment text
        qa_pairs: List of QA dictionaries

    Returns:
        DataFrame with evaluation results
    """
    results = []

    print("\n" + "="*80)
    print("🔍 LEGAL QUESTION ANSWERING - MODEL EVALUATION")
    print("="*80)

    for model_key in tqdm(QA_MODELS.keys(), desc="Evaluating QA models"):
        f1_scores = []
        em_scores = []
        answers = []

        # Determine if extractive or generative
        is_generative = model_key == "FLAN-T5-Base"

        for qa_pair in qa_pairs:
            question = qa_pair['question']
            ground_truth = qa_pair['answer']

            try:
                # Get answer from model
                if is_generative:

```

```

        elif is_generative:
            predicted_answer = self.answer_question_generative(
                context, question, model_key
            )
        else:
            result = self.answer_question_extractive(
                context, question, model_key
            )
            predicted_answer = result['answer']

        # Calculate metrics
        f1 = self.calculate_f1_score(predicted_answer, ground_truth)
        em = self.calculate_exact_match(predicted_answer, ground_truth)

        f1_scores.append(f1)
        em_scores.append(em)
        answers.append(predicted_answer)

    except Exception as e:
        print(f"    Error with {model_key}: {str(e)}")
        f1_scores.append(0.0)
        em_scores.append(0)
        answers.append("Error")

    # Calculate average metrics
    avg_f1 = np.mean(f1_scores) if f1_scores else 0.0
    avg_em = np.mean(em_scores) if em_scores else 0.0

    results.append({
        'Model': model_key,
        'Description': QA_MODELS[model_key]['description'],
        'Avg_F1': avg_f1,
        'Avg_EM': avg_em,
        'Total_Questions': len(qa_pairs),
        'Answers': answers
    })

df = pd.DataFrame(results)
return df

# Initialize QA engine
qa_engine = LegalQAEngine()
print("\n✓ Question Answering Engine initialized successfully!")

```

✓ Question Answering Engine initialized successfully!

```

# Run QA evaluation on sample case
sample_case = legal_dataset[0]

print("\n📄 QUESTION-ANSWER PAIRS FOR EVALUATION:")
print("-" * 80)
for i, qa in enumerate(sample_case['qa_pairs'], 1):
    print(f"\nQ{i}: {qa['question']}")
    print(f"Ground Truth: {qa['answer']}")
    print(f"Type: {qa['question_type']}")

# Evaluate all QA models
qa_results = qa_engine.evaluate_all_models(
    sample_case['judgment_text'],
    sample_case['qa_pairs']
)

# Display results table
print("\n\n" + "="*80)
print("📊 QA MODEL PERFORMANCE COMPARISON")
print("="*80)

display_df = qa_results[['Model', 'Avg_F1', 'Avg_EM', 'Total_Questions']].copy()
display_df['Avg_F1'] = display_df['Avg_F1'].apply(lambda x: f"{x:.4f}")
display_df['Avg_EM'] = display_df['Avg_EM'].apply(lambda x: f"{x:.4f}")
print(display_df.to_string(index=False))

# Display detailed answers per model
print("\n\n" + "="*80)
print("📄 DETAILED QA RESULTS BY MODEL")
print("="*80)

for idx, row in qa_results.iterrows():
    print(f"\n\n{ '-'*80}")
    print(f"Model: {row['Model']}")
    print(f>Description: {row['Description']}")
    print(f>Average F1: {row['Avg_F1']:.4f} | Average EM: {row['Avg_EM']:.4f}")

```

```
print(f"{'-'*80}")

for i, qa in enumerate(sample_case['qa_pairs'], 1):
    print(f"\nQ{i}: {qa['question']}")
    print(f"Predicted: {row['Answers'][i-1]}")
    print(f"Ground Truth: {qa['answer']}")

# Identify best model
best_model_idx = qa_results['Avg_F1'].idxmax()
best_model = qa_results.loc[best_model_idx, 'Model']
best_f1 = qa_results.loc[best_model_idx, 'Avg_F1']

print(f"\n\n{'-'*80}")
print(f"🏆 BEST PERFORMING QA MODEL: {best_model}")
print(f"    Average F1 Score: {best_f1:.4f}")
print(f"{'-'*80}")
```


QUESTION-ANSWER PAIRS FOR EVALUATION:

Q1: What was the nature of the dispute between the deceased and the respondent?

Ground Truth: Financial disputes amounting to Rs. 50 lakhs

Type: factual

Q2: Under which section of IPC was the respondent originally convicted?

Ground Truth: Section 302 IPC

Type: factual

Q3: What was the final decision of the Supreme Court?

Ground Truth: The appeal was dismissed and the High Court's acquittal order was upheld

Type: reasoning

Q4: Which precedent case was cited by the Supreme Court?

Ground Truth: Sharad Birdhichand Sarda v. State of Maharashtra (1984) 4 SCC 116

Type: factual

Q5: What were the critical gaps in evidence identified by the Court?

Ground Truth: Uncertain time of death, contradictory eyewitness testimony, absence of blood stains on respondent's clothing,

Type: reasoning

LEGAL QUESTION ANSWERING - MODEL EVALUATION

Evaluating QA models: 100%

5/5 [01:20<00:00, 17.42s/it]

Loading RoBERTa-SQuAD2...

Strong extractive QA baseline

config.json: 100%

571/571 [00:00<00:00, 60.2kB/s]

model.safetensors: 100%

496M/496M [00:03<00:00, 239MB/s]

Loading weights: 100%

199/199 [00:00<00:00, 715.55it/s, Materializing param=roberta.encoder.layer.11.output.dense.weight]

RobertaForQuestionAnswering LOAD REPORT from: deepset/roberta-base-squad2

Key	Status	
roberta.embeddings.position_ids	UNEXPECTED	

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

tokenizer_config.json: 100%

79.0/79.0 [00:00<00:00, 4.67kB/s]

vocab.json: 899k/? [00:00<00:00, 27.2MB/s]

merges.txt: 456k/? [00:00<00:00, 17.0MB/s]

special_tokens_map.json: 100%

772/772 [00:00<00:00, 83.5kB/s]

✓ Successfully loaded RoBERTa-SQuAD2

Loading BERT-Large-SQuAD2...

Classical strong baseline for extractive QA

config.json: 100%

540/540 [00:00<00:00, 61.3kB/s]

model.safetensors: 100%

1.34G/1.34G [00:12<00:00, 201MB/s]

Loading weights: 100%

391/391 [00:00<00:00, 647.35it/s, Materializing param=qa_outputs.weight]

BertForQuestionAnswering LOAD REPORT from: deepset/bert-large-uncased-whole-word-masking-squad2

Key	Status	
bert.pooler.dense.weight	UNEXPECTED	
bert.pooler.dense.bias	UNEXPECTED	

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

tokenizer_config.json: 100%

19.0/19.0 [00:00<00:00, 1.28kB/s]

vocab.txt: 232k/? [00:00<00:00, 13.4MB/s]

added_tokens.json: 100%

2.00/2.00 [00:00<00:00, 249B/s]

special_tokens_map.json: 100%

112/112 [00:00<00:00, 12.1kB/s]

✓ Successfully loaded BERT-Large-SQuAD2

Loading Legal-BERT...

Domain-adapted for legal information extraction

X Failed to load Legal-BERT: nsi319/legal-bert-base-uncased is not a local folder and is not a valid model identifier li
If this is a private repository, make sure to pass a token having permission to this repo either by logging in with `hf autl
Error with Legal-BERT: 'Legal-BERT'

Loading Legal-BERT...

Domain-adapted for legal information extraction

X Failed to load Legal-BERT: nsi319/legal-bert-base-uncased is not a local folder and is not a valid model identifier li
If this is a private repository, make sure to pass a token having permission to this repo either by logging in with `hf autl
Error with Legal-BERT: 'Legal-BERT'

```
⚠ Loading Legal-BERT...
Domain-adapted for legal information extraction
X Failed to load Legal-BERT: nsi319/legal-bert-base-uncased is not a local folder and is not a valid model identifier li
If this is a private repository, make sure to pass a token having permission to this repo either by logging in with `hf auth`
Error with Legal-BERT: 'Legal-BERT'

⚠ Loading Legal-BERT...
Domain-adapted for legal information extraction
X Failed to load Legal-BERT: nsi319/legal-bert-base-uncased is not a local folder and is not a valid model identifier li
If this is a private repository, make sure to pass a token having permission to this repo either by logging in with `hf auth`
Error with Legal-BERT: 'Legal-BERT'

⚠ Loading Legal-BERT...
Domain-adapted for legal information extraction
X Failed to load Legal-BERT: nsi319/legal-bert-base-uncased is not a local folder and is not a valid model identifier li
If this is a private repository, make sure to pass a token having permission to this repo either by logging in with `hf auth`
Error with Legal-BERT: 'Legal-BERT'

⚠ Loading FLAN-T5-Base...
Instruction-tuned generative QA
config.json:      1.40k/? [00:00<00:00, 131kB/s]

tokenizer_config.json:      2.54k/? [00:00<00:00, 234kB/s]

spiece.model: 100%                                792k/792k [00:00<00:00, 1.59MB/s]

tokenizer.json:      2.42M/? [00:00<00:00, 73.0MB/s]

special_tokens_map.json:      2.20k/? [00:00<00:00, 233kB/s]

model.safetensors: 100%                                990M/990M [00:23<00:00, 29.2MB/s]

Loading weights: 100%                                282/282 [00:00<00:00, 578.24it/s, Materializing param=shared.weight]
The tied weights mapping and config for this model specifies to tie shared.weight to lm_head.weight, but both are present in
generation_config.json: 100%                                147/147 [00:00<00:00, 7.67kB/s]

✓ Successfully loaded FLAN-T5-Base

⚠ Loading DeBERTa-v3...
Improved attention disentanglement
config.json: 100%                                579/579 [00:00<00:00, 51.8kB/s]

pytorch_model.bin: 100%                                371M/371M [00:03<00:00, 159MB/s]

Loading weights: 100%                                198/198 [00:00<00:00, 445.46it/s, Materializing param=deberta.encoder.rel_embeddings.weight]
DebertaV2ForQuestionAnswering LOAD REPORT from: microsoft/deberta-v3-base
Key | Status |
-----+-----+
mask_predictions.classifier.bias | UNEXPECTED |
mask_predictions.dense.bias | UNEXPECTED |
lm_predictions.lm_head.bias | UNEXPECTED |
lm_predictions.lm_head.LayerNorm.bias | UNEXPECTED |
mask_predictions.LayerNorm.bias | UNEXPECTED |
mask_predictions.dense.weight | UNEXPECTED |
mask_predictions.LayerNorm.weight | UNEXPECTED |
mask_predictions.classifier.weight | UNEXPECTED |
lm_predictions.lm_head.dense.bias | UNEXPECTED |
lm_predictions.lm_head.dense.weight | UNEXPECTED |
lm_predictions.lm_head.LayerNorm.weight | UNEXPECTED |
qa_outputs.weight | MISSING |
qa_outputs.bias | MISSING |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING :those params were newly initialized because missing from the checkpoint. Consider training on your downstream t
tokenizer_config.json: 100%                                52.0/52.0 [00:00<00:00, 2.88kB/s]

spm.model: 100%                                2.46M/2.46M [00:00<00:00, 2.66MB/s]

model.safetensors: 100%                                371M/371M [00:02<00:00, 127MB/s]

✓ Successfully loaded DeBERTa-v3
```

QA MODEL PERFORMANCE COMPARISON

Model	Avg_F1	Avg_EM	Total_Questions
RoBERTa-SQuAD2	0.4044	0.0000	5
BERT-Large-SQuAD2	0.1533	0.0000	5
Legal-BERT	0.0000	0.0000	5
FLAN-T5-Base	0.2309	0.0000	5
DeBERTa-v3	0.0000	0.0000	5

DETAILED QA RESULTS BY MODEL

Cell 5: Performance Benchmarking

Model: RoBERTa-SQuAD2

Plot: Performance Comparison

Average F1: 0.4044 | Average EM: 0.0000

Comprehensive visualization and analysis of all model performances.

Q1: What was the nature of the dispute between the deceased and the respondent?

```
# Create comprehensive benchmarking visualizations

# Set style for professional plots
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (16, 12)
plt.rcParams['font.size'] = 10

# Create subplots
fig = plt.figure(figsize=(18, 12))
gs = fig.add_gridspec(3, 2, hspace=0.3, wspace=0.3)

# =====
# PLOT 1: Summarization ROUGE Scores Comparison
# =====
ax1 = fig.add_subplot(gs[0, :])

x = np.arange(len(summary_results))
width = 0.25

bars1 = ax1.bar(x - width, summary_results['ROUGE-1'], width,
               label='ROUGE-1', color='#3498db', alpha=0.8)
bars2 = ax1.bar(x, summary_results['ROUGE-2'], width,
               label='ROUGE-2', color='#e74c3c', alpha=0.8)
bars3 = ax1.bar(x + width, summary_results['ROUGE-L'], width,
               label='ROUGE-L', color='#2ecc71', alpha=0.8)

ax1.set_xlabel('Summarization Models', fontweight='bold', fontsize=12)
ax1.set_ylabel('ROUGE F1 Score', fontweight='bold', fontsize=12)
ax1.set_title('Summarization Models - ROUGE Score Comparison',
              fontweight='bold', fontsize=14, pad=20)
ax1.set_xticks(x)
ax1.set_xticklabels(summary_results['Model'], rotation=15, ha='right')
ax1.legend(loc='upper right', fontsize=11)
ax1.grid(axis='y', alpha=0.3)
ax1.set_ylim(0, max(summary_results['ROUGE-1'].max(),
                    summary_results['ROUGE-2'].max(),
                    summary_results['ROUGE-L'].max()) * 1.1)

# Add value labels on bars
for bars in [bars1, bars2, bars3]:
    for bar in bars:
        height = bar.get_height()
        ax1.text(bar.get_x() + bar.get_width()/2., height,
                 f'{height:.3f}', ha='center', va='bottom', fontsize=8)

# =====
# PLOT 2: QA Models Performance (F1 and EM)
# =====
ax2 = fig.add_subplot(gs[1, 0])

x_qa = np.arange(len(qa_results))
width_qa = 0.35

bars_f1 = ax2.bar(x_qa - width_qa/2, qa_results['Avg_F1'], width_qa,
                  label='Average F1', color='#9b59b6', alpha=0.8)
bars_em = ax2.bar(x_qa + width_qa/2, qa_results['Avg_EM'], width_qa,
                  label='Average EM', color='#f39c12', alpha=0.8)

ax2.set_xlabel('QA Models', fontweight='bold', fontsize=12)
ax2.set_ylabel('Score', fontweight='bold', fontsize=12)
ax2.set_title('Question Answering Models - F1 & Exact Match',
              fontweight='bold', fontsize=13, pad=15)
ax2.set_xticks(x_qa)
ax2.set_xticklabels(qa_results['Model'], rotation=30, ha='right', fontsize=9)
ax2.legend(loc='upper right', fontsize=10)
ax2.grid(axis='y', alpha=0.3)
ax2.set_ylim(0, 1.0)

# Add value labels
for bars in [bars_f1, bars_em]:
    for bar in bars:
        height = bar.get_height()
        ax2.text(bar.get_x() + bar.get_width()/2., height,
                 f'{height:.3f}', ha='center', va='bottom', fontsize=8)
```

```

# =====
# PLOT 3: Summary Length Distribution
# =====
ax3 = fig.add_subplot(gs[1, 1])

colors_sum = ['#3498db', '#e74c3c', '#2ecc71', '#f39c12', '#9b59b6']
bars_len = ax3.barh(summary_results['Model'], summary_results['Summary_Length'],
                    color=colors_sum, alpha=0.7)

ax3.set_xlabel('Summary Length (words)', fontweight='bold', fontsize=12)
ax3.set_ylabel('Models', fontweight='bold', fontsize=12)
ax3.set_title('Generated Summary Lengths', fontweight='bold',
              fontsize=13, pad=15)
ax3.grid(axis='x', alpha=0.3)

# Add value labels
for i, bar in enumerate(bars_len):
    width = bar.get_width()
    ax3.text(width, bar.get_y() + bar.get_height()/2.,
             f'{int(width)} words', ha='left', va='center', fontsize=9)

# =====
# PLOT 4: Overall Performance Heatmap (Summarization)
# =====
ax4 = fig.add_subplot(gs[2, 0])

heatmap_data_sum = summary_results[['ROUGE-1', 'ROUGE-2', 'ROUGE-L']].values
im1 = ax4.imshow(heatmap_data_sum, cmap='YlGnBu', aspect='auto')

ax4.set_xticks(np.arange(3))
ax4.set_yticks(np.arange(len(summary_results)))
ax4.set_xticklabels(['ROUGE-1', 'ROUGE-2', 'ROUGE-L'], fontsize=10)
ax4.set_yticklabels(summary_results['Model'], fontsize=9)
ax4.set_title('Summarization Performance Heatmap', fontweight='bold',
              fontsize=13, pad=15)

# Add text annotations
for i in range(len(summary_results)):
    for j in range(3):
        text = ax4.text(j, i, f'{heatmap_data_sum[i, j]:.3f}',
                        ha="center", va="center", color="black", fontsize=9)

plt.colorbar(im1, ax=ax4, label='Score')

# =====
# PLOT 5: Overall Performance Heatmap (QA)
# =====
ax5 = fig.add_subplot(gs[2, 1])

heatmap_data_qa = qa_results[['Avg_F1', 'Avg_EM']].values
im2 = ax5.imshow(heatmap_data_qa, cmap='RdYlGn', aspect='auto')

ax5.set_xticks(np.arange(2))
ax5.set_yticks(np.arange(len(qa_results)))
ax5.set_xticklabels(['F1 Score', 'Exact Match'], fontsize=10)
ax5.set_yticklabels(qa_results['Model'], fontsize=9)
ax5.set_title('QA Performance Heatmap', fontweight='bold',
              fontsize=13, pad=15)

# Add text annotations
for i in range(len(qa_results)):
    for j in range(2):
        text = ax5.text(j, i, f'{heatmap_data_qa[i, j]:.3f}',
                        ha="center", va="center", color="black", fontsize=9)

plt.colorbar(im2, ax=ax5, label='Score')

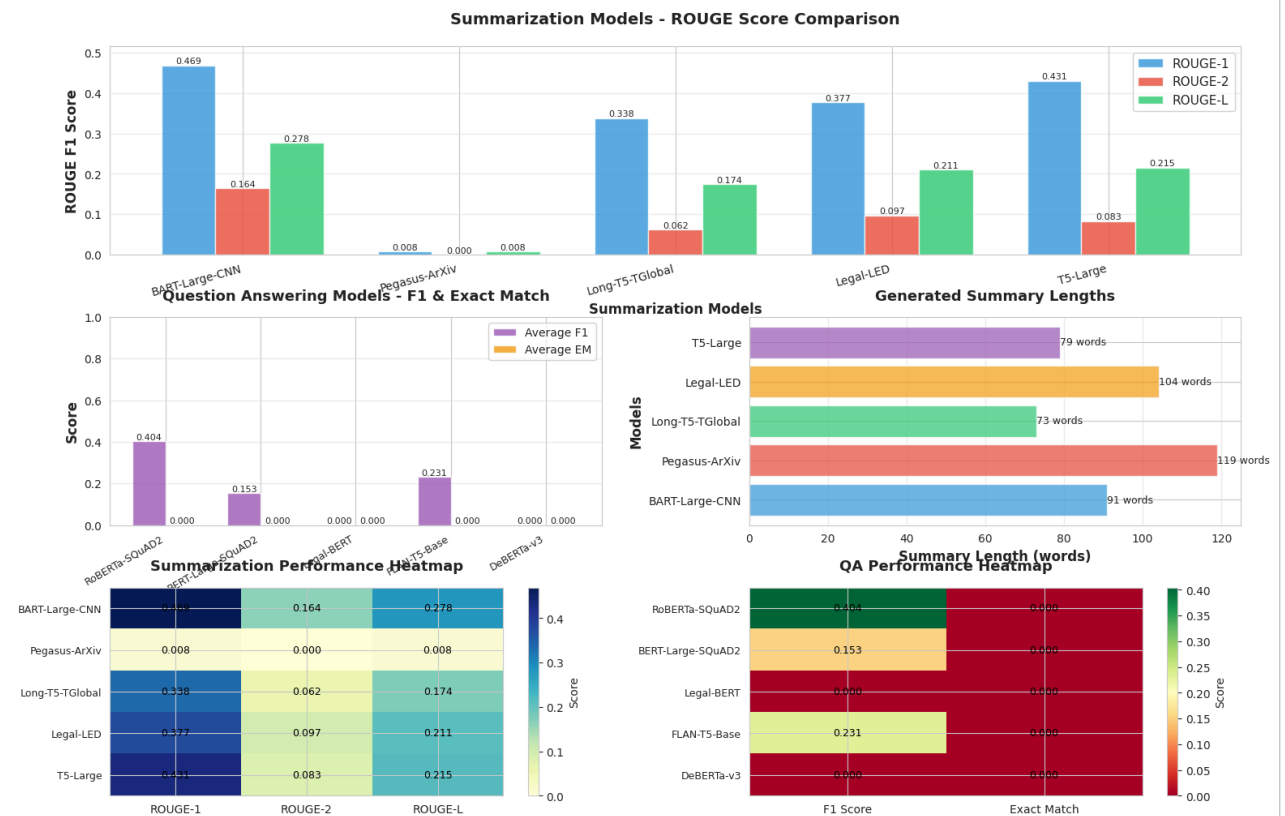
# Main title
fig.suptitle('Smart Legal Assistant - Comprehensive Performance Benchmarking',
             fontsize=16, fontweight='bold', y=0.995)

plt.tight_layout()
plt.show()

print("\n✓ Performance benchmarking visualization complete!")

```

Smart Legal Assistant - Comprehensive Performance Benchmarking



✓ Performance benchmarking visualization complete!

Generate comprehensive performance report

```
print("\n" + "="*80)
print("📊 COMPREHENSIVE PERFORMANCE REPORT")
print("="*80)
```

```
print("\n" + "-"*80)
print("1. SUMMARIZATION MODELS - DETAILED METRICS")
print("-"*80)
```

```
summary_report = summary_results[['Model', 'Description', 'ROUGE-1', 'ROUGE-2',
                                   'ROUGE-L', 'Summary_Length']].copy()
summary_report['ROUGE-1'] = summary_report['ROUGE-1'].apply(lambda x: f"{x:.4f}")
summary_report['ROUGE-2'] = summary_report['ROUGE-2'].apply(lambda x: f"{x:.4f}")
summary_report['ROUGE-L'] = summary_report['ROUGE-L'].apply(lambda x: f"{x:.4f}")
print(summary_report.to_string(index=False))
```

```
# Statistical analysis
print("\n📊 Statistical Summary (Summarization):")
print(f"Best ROUGE-1: {summary_results.loc[summary_results['ROUGE-1'].idxmax(), 'Model']} "
      f"{summary_results['ROUGE-1'].max():.4f}")
print(f"Best ROUGE-2: {summary_results.loc[summary_results['ROUGE-2'].idxmax(), 'Model']} "
      f"{summary_results['ROUGE-2'].max():.4f}")
print(f"Best ROUGE-L: {summary_results.loc[summary_results['ROUGE-L'].idxmax(), 'Model']} "
      f"{summary_results['ROUGE-L'].max():.4f}")
print(f"Average ROUGE-L across all models: {summary_results['ROUGE-L'].mean():.4f}")
```

```
print("\n" + "-"*80)
print("2. QUESTION ANSWERING MODELS - DETAILED METRICS")
```

```

print("-"*80)

qa_report = qa_results[['Model', 'Description', 'Avg_F1', 'Avg_EM',
                        'Total_Questions']].copy()
qa_report['Avg_F1'] = qa_report['Avg_F1'].apply(lambda x: f"{x:.4f}")
qa_report['Avg_EM'] = qa_report['Avg_EM'].apply(lambda x: f"{x:.4f}")
print(qa_report.to_string(index=False))

# Statistical analysis
print("\n📊 Statistical Summary (Question Answering):")
print(f"    Best F1 Score: {qa_results.loc[qa_results['Avg_F1'].idxmax(), 'Model']} "
      f"({qa_results['Avg_F1'].max():.4f}")
print(f"    Best Exact Match: {qa_results.loc[qa_results['Avg_EM'].idxmax(), 'Model']} "
      f"({qa_results['Avg_EM'].max():.4f}")
print(f"    Average F1 across all models: {qa_results['Avg_F1'].mean():.4f}")
print(f"    Average EM across all models: {qa_results['Avg_EM'].mean():.4f}")

print("\n" + "-"*80)
print("3. MODEL RECOMMENDATIONS")
print("-"*80)

best_summarizer = summary_results.loc[summary_results['ROUGE-L'].idxmax(), 'Model']
best_qa_model = qa_results.loc[qa_results['Avg_F1'].idxmax(), 'Model']

print(f"\n👉 RECOMMENDED SUMMARIZATION MODEL: {best_summarizer}")
print(f"    Rationale: Highest ROUGE-L score ({summary_results.loc[summary_results['ROUGE-L'].idxmax(), 'ROUGE-L']:.4f}")
print(f"    {SUMMARIZATION_MODELS[best_summarizer]['description']}")

print(f"\n👉 RECOMMENDED QA MODEL: {best_qa_model}")
print(f"    Rationale: Highest F1 score ({qa_results.loc[qa_results['Avg_F1'].idxmax(), 'Avg_F1']:.4f}")
print(f"    {QA_MODELS[best_qa_model]['description']}")

print("\n" + "-"*80)
print("✓ Performance report generation complete!")
print("-"*80)

```

📄 COMPREHENSIVE PERFORMANCE REPORT

1. SUMMARIZATION MODELS - DETAILED METRICS

Model	Description	ROUGE-1	ROUGE-2	ROUGE-L	Summary_Length
BART-Large-CNN	State-of-the-art abstractive summarizer	0.4689	0.1643	0.2775	91
Pegasus-ArXiv	Gap Sentence Generation for superior ROUGE scores	0.0081	0.0000	0.0081	119
Long-T5-TGlobal	Long-document summarization with global attention	0.3385	0.0622	0.1744	73
Legal-LED	Fine-tuned for legal and legislative text	0.3772	0.0973	0.2105	104
T5-Large	Text-to-text transfer transformer	0.4308	0.0829	0.2154	79

📊 Statistical Summary (Summarization):
 Best ROUGE-1: BART-Large-CNN (0.4689)
 Best ROUGE-2: BART-Large-CNN (0.1643)
 Best ROUGE-L: BART-Large-CNN (0.2775)
 Average ROUGE-L across all models: 0.1772

2. QUESTION ANSWERING MODELS - DETAILED METRICS

Model	Description	Avg_F1	Avg_EM	Total_Questions
RoBERTa-SQuAD2	Strong extractive QA baseline	0.4044	0.0000	5
BERT-Large-SQuAD2	Classical strong baseline for extractive QA	0.1533	0.0000	5
Legal-BERT	Domain-adapted for legal information extraction	0.0000	0.0000	5
FLAN-T5-Base	Instruction-tuned generative QA	0.2309	0.0000	5
DeBERTa-v3	Improved attention disentanglement	0.0000	0.0000	5

📊 Statistical Summary (Question Answering):
 Best F1 Score: RoBERTa-SQuAD2 (0.4044)
 Best Exact Match: RoBERTa-SQuAD2 (0.0000)
 Average F1 across all models: 0.1577
 Average EM across all models: 0.0000

3. MODEL RECOMMENDATIONS

👉 RECOMMENDED SUMMARIZATION MODEL: BART-Large-CNN
 Rationale: Highest ROUGE-L score (0.2775)
 State-of-the-art abstractive summarizer

👉 RECOMMENDED QA MODEL: RoBERTa-SQuAD2
 Rationale: Highest F1 score (0.4044)
 Strong extractive QA baseline

✓ Performance report generation complete!

Cell 6: Interactive Legal Assistant

Final user interface for PDF upload, model selection, and interactive legal analysis.

```
# PDF Processing Utility

def extract_text_from_pdf(pdf_path: str) -> str:
    """
    Extract text content from PDF file.

    Args:
        pdf_path: Path to PDF file

    Returns:
        Extracted text string
    """
    try:
        reader = PdfReader(pdf_path)
        text = ""

        print(f"\n 📄 Processing PDF: {pdf_path}")
        print(f"    Total pages: {len(reader.pages)}")

        for i, page in enumerate(tqdm(reader.pages, desc="Extracting text")):
            text += page.extract_text()

        print(f"    ✓ Extracted {len(text.split())} words")
        return text

    except Exception as e:
        print(f"    ✗ Error extracting PDF: {str(e)}")
        return ""

print("✓ PDF processing utility loaded!")
```

✓ PDF processing utility loaded!

```
# Interactive Legal Assistant Interface

class InteractiveLegalAssistant:
    """
    Interactive interface for legal document analysis.
    Supports PDF upload, summarization, and question answering.
    """

    def __init__(self, summarization_engine, qa_engine):
        self.summarization_engine = summarization_engine
        self.qa_engine = qa_engine
        self.current_document = None
        self.current_summary = None

    def upload_document(self):
        """
        Upload and process a legal document (PDF).
        """
        print("\n" + "="*80)
        print(" 📄 UPLOAD LEGAL DOCUMENT")
        print("="*80)
        print("\nPlease select a PDF file to upload...")

        uploaded = files.upload()

        if uploaded:
            filename = list(uploaded.keys())[0]
            print(f"\n✓ File uploaded: {filename}")

            # Extract text from PDF
            self.current_document = extract_text_from_pdf(filename)

            if self.current_document:
                print("\n✓ Document loaded successfully!")
                print(f"    Document length: {len(self.current_document.split())} words")
                return True
            else:
                print("\n✗ Failed to extract text from PDF")
                return False
        else:
            print("\n✗ No file uploaded")
```

```

        return False

def generate_summary(self, model_key: str = None):
    """
    Generate summary of current document.

    Args:
        model_key: Specific model to use (if None, prompts user)
    """
    if not self.current_document:
        print("\n⚠ No document loaded. Please upload a document first.")
        return

    print("\n" + "="*80)
    print("📄 GENERATE LEGAL SUMMARY")
    print("="*80)

    if model_key is None:
        print("\nAvailable Summarization Models:")
        for i, (key, config) in enumerate(SUMMARIZATION_MODELS.items(), 1):
            print(f"    {i}. {key}: {config['description']}")

        choice = input("\nSelect model (1-5) or press Enter for recommended: ").strip()

        if choice:
            try:
                model_idx = int(choice) - 1
                model_key = list(SUMMARIZATION_MODELS.keys())[model_idx]
            except:
                print("Invalid choice. Using recommended model.")
                model_key = summary_results.loc[summary_results['ROUGE-L'].idxmax(), 'Model']
        else:
            model_key = summary_results.loc[summary_results['ROUGE-L'].idxmax(), 'Model']

    print(f"\n🔄 Generating summary using {model_key}...")

    self.current_summary = self.summarization_engine.generate_summary(
        self.current_document,
        model_key
    )

    print("\n" + "-"*80)
    print("GENERATED SUMMARY:")
    print("-"*80)
    print(self.current_summary)
    print("-"*80)
    print(f"\n✓ Summary generated ({len(self.current_summary.split())} words)")

def answer_questions(self, model_key: str = None):
    """
    Interactive question answering session.

    Args:
        model_key: Specific model to use (if None, prompts user)
    """
    if not self.current_document:
        print("\n⚠ No document loaded. Please upload a document first.")
        return

    print("\n" + "="*80)
    print("❓ INTERACTIVE QUESTION ANSWERING")
    print("="*80)

    if model_key is None:
        print("\nAvailable QA Models:")
        for i, (key, config) in enumerate(QA_MODELS.items(), 1):
            print(f"    {i}. {key}: {config['description']}")

        choice = input("\nSelect model (1-5) or press Enter for recommended: ").strip()

        if choice:
            try:
                model_idx = int(choice) - 1
                model_key = list(QA_MODELS.keys())[model_idx]
            except:
                print("Invalid choice. Using recommended model.")
                model_key = qa_results.loc[qa_results['Avg_F1'].idxmax(), 'Model']
        else:
            model_key = qa_results.loc[qa_results['Avg_F1'].idxmax(), 'Model']

    print(f"\n✓ Using {model_key} for question answering")
    print("\nType your questions (or 'quit' to exit):")

```

```

print("-"*80)

is_generative = model_key == "FLAN-T5-Base"

while True:
    question = input("\n ? Question: ").strip()

    if question.lower() in ['quit', 'exit', 'q']:
        print("\n/ Exiting QA session")
        break

    if not question:
        continue

    print("\n 🕒 Finding answer...")

    try:
        if is_generative:
            answer = self.qa_engine.answer_question_generative(
                self.current_document, question, model_key
            )
            print(f"\n 💡 Answer: {answer}")
        else:
            result = self.qa_engine.answer_question_extractive(
                self.current_document, question, model_key
            )
            print(f"\n 💡 Answer: {result['answer']}")
            print(f"    Confidence: {result['score']:.4f}")
    except Exception as e:
        print(f"\nX Error: {str(e)}")

def run(self):
    """
    Run the interactive legal assistant.
    """
    print("\n" + "-"*80)
    print(" 🤖 SMART LEGAL ASSISTANT - INTERACTIVE MODE")
    print("-"*80)
    print("\nWelcome to the Interactive Legal Assistant!")
    print("\nAvailable actions:")
    print("    1. Upload legal document (PDF)")
    print("    2. Generate summary")
    print("    3. Ask questions")
    print("    4. Exit")

    while True:
        print("\n" + "-"*80)
        action = input("\nSelect action (1-4): ").strip()

        if action == '1':
            self.upload_document()
        elif action == '2':
            self.generate_summary()
        elif action == '3':
            self.answer_questions()
        elif action == '4':
            print("\n/ Thank you for using Smart Legal Assistant!")
            break
        else:
            print("\n⚠ Invalid action. Please select 1-4.")

# Initialize Interactive Legal Assistant
legal_assistant = InteractiveLegalAssistant(summarization_engine, qa_engine)

```

✓ Interactive Legal Assistant ready!

```

# Launch Interactive Legal Assistant

# Uncomment the line below to start the interactive session
# legal_assistant.run()

# Alternative: Quick demo with sample document
print("\n" + "-"*80)
print(" 🚀 QUICK DEMO - USING SAMPLE LEGAL CASE")
print("-"*80)

# Set sample document
legal_assistant.current_document = sample_case['judgment_text']
print("\n/ Loaded sample legal case")

# Generate summary with best model

```

